

RECURSION TREE VISUALIZER

First Phase Project Report

Design and Analysis of Algorithms (DAA)

Web-Based Visualization Tool for Divide-and-Conquer Algorithms

Algorithms Covered:

Karatsuba Multiplication · Maximum Subarray (Divide & Conquer)

Hosted at:

<https://algorithm-visualizer-kndma.vercel.app/>

Submitted By:

Abhinav Sharma(24BCT0116) and Aayan Mohanty(24BKT0136)

Submitted for academic evaluation · Original work

1.0 Introduction

One of the persistent challenges in teaching algorithms is bridging the gap between a written algorithm and a felt understanding of how it executes. This is especially true for recursive algorithms, where the dynamic unfolding of nested calls, sub-problem decompositions, and returning values is difficult to internalize from pseudocode or static diagrams alone. A student can read the recurrence $T(n) = 3T(n/2) + O(n)$ and compute its solution, but that computation rarely produces the intuitive grasp of why the structure works the way it does, what the three subproblems look like, how deep the recursion goes, and how results combine on the way back up.

The Recursion Tree Visualizer is a web-based educational tool developed as a project for the Design and Analysis of Algorithms (DAA) course. It addresses this gap directly by rendering the execution of recursive algorithms as interactive, animated, step-by-step tree structures and making the mechanics of recursion visible, navigable, and analytically transparent. The tool is hosted at <https://algorithm-visualizer-kndma.vercel.app/> and requires no installation, running entirely in the browser on any device.

Two divide-and-conquer algorithms are currently implemented. The first is Karatsuba Multiplication, a fast integer multiplication algorithm first published by Anatoly Karatsuba in 1962, whose sub-quadratic $O(n^{1.585})$ complexity arises from a deceptively simple insight: reducing four recursive subproblems to three through an algebraic rearrangement. This reduction is most naturally understood not from the formula but from looking at the recursion tree, where the ternary branching factor of 3 stands in direct contrast to the 4-way branching of the naïve approach. The second algorithm is the Maximum Subarray problem solved via divide-and-conquer, whose binary recursive decomposition produces a clean $\log n$ -deep tree

that makes the $O(n \log n)$ complexity class visually immediate. Together, the two algorithms allow students to place a ternary tree beside a binary tree and directly observe how branching factor and recurrence structure determine complexity class.

The tool supports step-by-step forward and backward navigation through the recursion, an algorithm selector for switching between the two implementations, and a reset function for repeated study. The interface is intentionally minimal, keeping focus entirely on the tree structure and the algorithmic relationships it encodes. This design choice is grounded in a body of CS education research showing that actively navigable visualizations produce measurably better learning outcomes than either passive animations or textual descriptions alone.

1.1 Problem Statement

Recursion and divide-and-conquer strategies are central to the Design and Analysis of Algorithms curriculum, yet students consistently report them among the most difficult topics to internalize. The core difficulty is representational: the standard tools available to students describe algorithmic structure but do not make it visible. When a student reads that Karatsuba splits a multiplication into three subproblems, they understand the fact but typically do not build a mental model of what those three calls look like, how many total nodes the recursion tree contains, or how the depth of the tree changes with input size. The Recursion Tree Visualizer directly addresses this representational gap.

1.2 Objectives

1. To visualize the step-by-step execution of Karatsuba Multiplication and the Maximum Subarray divide-and-conquer algorithm as animated, navigable recursion trees.
2. To help students understand the relationship between an algorithm's recurrence structure and its asymptotic complexity by making the tree's depth and branching factor directly observable.

3. To provide interactive navigation that enables self-paced, active exploration rather than passive viewing.
4. To deliver these capabilities through a zero-installation, browser-based interface accessible without any setup or dependencies.
5. To serve as a supplementary learning tool for DAA coursework, examination preparation, and classroom demonstration.

1.3 Modifications from Abstract

The original project abstract proposed the visualization of a broader set of algorithms including Merge Sort and the Fibonacci sequence. Based on feedback received and a reassessment of scope, the implementation was focused on two algorithms that together provide a more pedagogically coherent comparison: both are divide-and-conquer algorithms of non-trivial complexity whose key properties are most clearly expressed through their recursion trees, and whose side-by-side placement (ternary vs. binary branching) provides genuine comparative insight. The inclusion of Merge Sort and Fibonacci remains a planned future extension, as noted in the conclusion.

2.0 Literature Review

ABSTRACT

This literature review examines the research foundations underlying the design and educational rationale of the Recursion Tree Visualizer, a web-based tool for animating the execution of divide-and-conquer algorithms in a Design and Analysis of Algorithms (DAA) course context. The review is structured across five sections: an introduction establishing background and research objectives; a methodology section detailing the search strategy, databases consulted, and inclusion/exclusion criteria applied to a candidate pool of 34 sources, from which 22 were retained; a thematic main body covering three intersecting research areas; a critical discussion; and a conclusion with implications for future work. The three themes examined are: (1) the empirical effectiveness of algorithm visualization tools in computer science education, drawing on meta-analytic evidence and recent experimental studies; (2) the specific pedagogical challenges of teaching recursion and the limitations of existing recursion visualization platforms including VisuAlgo, SRec, and Python Tutor; and (3) the algorithmic theory and educational significance of the two algorithms implemented, namely Karatsuba Multiplication and the Maximum Subarray divide-and-conquer formulation. The discussion identifies four substantive gaps in the existing literature and tooling landscape that the Recursion Tree Visualizer is positioned to address: the absence of a dedicated Karatsuba visualizer, the deployment friction of existing recursion tools, the lack of multi-algorithm comparative visualization, and the broader absence of longitudinal studies on recursion tree tools. The review concludes that the Recursion Tree Visualizer occupies a well-defined and currently underserved niche, providing purpose-built, browser-accessible, step-by-step recursion tree animation for two of the most instructive divide-and-conquer algorithms in the DAA curriculum and that its design decisions are grounded in a consistent body of empirical evidence on active engagement and visual learning in algorithm education.

Keywords: algorithm visualization, recursion tree, divide-and-conquer, Karatsuba multiplication, maximum subarray, DAA education, web-based learning tools

2.1 Introduction

2.1.1 Background and Context

The teaching of algorithms and data structures is widely regarded as one of the most cognitively demanding areas in undergraduate computer science education. Algorithms describe dynamic, abstract computational processes that resist comprehension through static, textual descriptions alone. This challenge is particularly acute for recursive algorithms, where students must simultaneously hold multiple nested call states in working memory while tracking how values propagate back up through the call chain. Over two decades of research

in computer science education has consistently found that algorithm visualization (AV) tools offer measurable pedagogical benefits over conventional instruction when used correctly. Hundhausen, Douglas, and Stasko (2002), in a landmark meta-study of 24 experimental studies, concluded that the most effective deployments of AV technology were those that actively engaged students through prediction tasks, what-if analyses, and interactive navigation rather than passive animation viewing. This distinction between active and passive engagement has since become a foundational principle in the design of educational algorithm tools.

The Recursion Tree Visualizer is a web-based application developed as part of a DAA course project. It renders the execution of two canonical divide-and-conquer algorithms, Karatsuba Multiplication and the Maximum Subarray problem as animated, navigable recursion trees. This literature review situates the project within the broader research landscape of algorithm visualization, recursion pedagogy, and divide-and-conquer education.

2.1.2 Scope of the Review

This review examines research across four intersecting domains: (1) the general effectiveness of algorithm visualization tools in CS education; (2) the specific pedagogical challenges of teaching recursion and divide-and-conquer strategies; (3) the algorithmic theory and educational literature pertaining to Karatsuba Multiplication and Maximum Subarray; and (4) design principles for web-based interactive learning environments. The review excludes graph algorithm visualization, sorting-only tools, studies focused on K–12 audiences, and purely theoretical algorithmic treatments lacking educational applicability.

2.1.3 Research Objectives

1. To establish the empirical basis for using visual and interactive tools in algorithm education, with a focus on recursion-specific tools.
2. To survey and critically evaluate existing algorithm visualization platforms to identify their capabilities and limitations relative to the Recursion Tree Visualizer.

3. To review the algorithmic and educational literature on Karatsuba Multiplication and the Maximum Subarray divide-and-conquer algorithm.
4. To identify gaps in existing tools and literature that the Recursion Tree Visualizer addresses.

2.2 Methodology (Search Strategy)

This literature review follows a structured search strategy designed to ensure coverage of relevant peer-reviewed, conference, and educational resource literature. The search was conducted between March 2025 and March 2026, drawing on academic databases, university course materials, and reputable educational platforms. A total of 34 sources were identified; after applying inclusion/exclusion criteria, 22 were retained.

2.2.1 Inclusion and Exclusion Criteria

Criterion	Details
Publication period	1990–2026; exceptions for seminal earlier works (Karatsuba 1962, Grenander 1977)
Language	English-language publications only
Source types	Peer-reviewed journals, ACM/IEEE proceedings, university course materials, established educational platforms
Relevance	Must directly address algorithm visualization, recursion pedagogy, divide-and-conquer algorithms, or web-based educational tool design
Excluded	K–12 studies; sorting/graph-only tools unrelated to recursion trees; non-English publications; grey literature without academic attribution

2.2.2 Search Databases and Keywords

The following databases and platforms were consulted: ACM Digital Library (SIGCSE, ITICSE proceedings); Google Scholar; Springer Nature and Elsevier ScienceDirect; SSRN; university course repositories (Cornell CS3110, MIT 6.006, UChicago CMSC-272); and educational platforms (VisuAlgo, GeeksforGeeks, Runestone Academy, Brilliant.org). Primary search strings included: "algorithm visualization" AND "education"; "recursion tree" AND "visualization"; "divide and conquer" AND "teaching"; "Karatsuba

multiplication" AND "complexity"; "maximum subarray" AND "divide and conquer" AND "Kadane"; "SRec" OR "VisuAlgo" OR "Python Tutor" AND "recursion".

2.3 Main Body

2.3.1 Theme 1: Effectiveness of Algorithm Visualization in CS Education

The use of visualization as a pedagogical tool in CS education has been studied extensively since the late 1980s. The earliest influential framework came from Brown and Sedgewick's BALSA system (1984), which pioneered the concept of software that animated algorithmic execution in real time. The most cited empirical foundation is the meta-study by Hundhausen, Douglas, and Stasko (2002), which aggregated findings from 24 experimental studies on AV technology. Their central finding was that the mode of use predicted learning outcomes. Students who engaged actively with visualizations by constructing animations, making predictions, or navigating step-by-step consistently outperformed those who passively watched pre-built animations. This finding directly justifies the step-by-step navigation design of the Recursion Tree Visualizer.

Hansen et al., in their evaluation of the HalVis hypermedia algorithm visualization system, reinforced these conclusions by comparing interactive AV tools against textbooks in controlled experiments. Students using HalVis showed significantly better performance on algorithm comprehension tests, attributed to the contextual immersion interactive tools provide. A 2024 study published in Educational Technology Research and Development (Springer) evaluated AVDS, a custom web-based animation platform for undergraduate Data Structures courses and found that approximately 95% of students using AVDS achieved grades above 80 on post-visualization assessments, with high user acceptance scores across both ease-of-use and perceived learning benefit dimensions. A VR-based algorithm visualization study cited by Academia.edu (2025) reported that 76.9% of students exposed to

interactive visualizations performed better on algorithmic tasks compared to control groups, confirming that interactivity is the critical ingredient.

2.3.2 Theme 2: Recursion Visualization — Challenges and Existing Tools

Recursion is consistently identified in the CS education literature as one of the most cognitively demanding undergraduate topics. Miller and Ranum, in their open textbook *Problem Solving with Algorithms and Data Structures* (Runestone Academy), devote a dedicated chapter to recursion visualization, noting that even students who grasp the mechanical rules of recursion often lack a stable mental model of what happens during execution. They advocate tree diagrams as the most effective representational form, noting that recursion trees make the stack of pending calls and the order of returns simultaneously visible. Research presented at SIGCSE 2023 found that teachers routinely draw recursion trees by hand during lectures because no adequate digital tool exists that renders these trees at the granularity needed for algorithm analysis, confirming that demand for automated, navigable recursion tree rendering is well-established but underserved.

Existing tools each address part of the problem. VisuAlgo (Halim, 2011–present) is the most comprehensive AV platform available and includes a recursion tree module, but covers generic recursive functions rather than specific DAA algorithms like Karatsuba, and lacks divide-and-conquer-specific annotations. SRec (Velázquez-Iturbide et al., 2008) is the most pedagogically grounded recursion-specific tool explicitly targeting Bloom's Taxonomy comprehension and analysis levels, but is a Java desktop application, creating deployment friction in modern educational environments. Python Tutor (Guo, 2013) is web-based and widely used, but operates at the code level (variable bindings, stack frames) rather than the algorithmic level, making it unsuitable for DAA-level analysis of divide-and-conquer decomposition. Velázquez-Iturbide et al. (2003) established the theoretical basis for automated recursion tree rendering, observing that graphical tree structure facilitates

orientation even when textual labels are dense supporting the architectural decision to make tree topology the primary visual element in this tool.

2.3.3 Theme 3: Karatsuba Multiplication and Maximum Subarray — Algorithm Theory and Educational Significance

Karatsuba Multiplication, published in 1962, was the first integer multiplication algorithm proven to be asymptotically faster than the classical $O(n^2)$ schoolbook method. The algorithm reduces four multiplications to three using the identity $(x_1y_2 + x_2y_1) = (x_1 + x_2)(y_1 + y_2) - x_1y_1 - x_2y_2$, yielding the recurrence $T(n) = 3T(n/2) + O(n)$ and complexity $O(n^{\log_2 3}) \approx O(n^{1.585})$. From an educational standpoint, it is a canonical case study in algorithmic improvement through algebraic cleverness. MIT 6.006 course materials emphasise the pedagogical contrast between the naïve 4-subproblem approach (remaining $O(n^2)$) and Karatsuba's 3-subproblem reduction, a contrast invisible in code but immediately apparent in the recursion tree's branching factor. The University of Chicago CMSC-272 handout notes the recurrence can be solved by direct induction, accessible to students not yet familiar with the full Master Theorem. Brilliant.org positions Karatsuba as a foundational example of divide-and-conquer achieving asymptotic improvement over historically established methods.

The Maximum Subarray problem, formally introduced by Grenander (1977) as a simplified model for maximum likelihood estimation has three principal algorithmic approaches spanning $O(n^2)$, $O(n \log n)$, and $O(n)$. Istiono (2023, SSRN) directly compared Kadane's algorithm against the divide-and-conquer approach and confirmed Kadane's practical runtime superiority. Despite this, the divide-and-conquer formulation is pedagogically indispensable: its recurrence $T(n) = 2T(n/2) + O(n)$, identical to Merge Sort's and produces $O(n \log n)$ through a canonical binary recursion structure that the recursion tree makes immediately visible. GeeksforGeeks's widely referenced treatment confirms that while Kadane's $O(n)$ algorithm is optimal for production, the divide-and-conquer approach is the

preferred pedagogical vehicle for demonstrating the $O(n \log n)$ complexity class. The problem also has applied relevance in genomic sequence analysis, financial time series, and computer vision.

2.4 Discussion

2.4.1 Summary of Existing Knowledge

The literature reviewed converges on several well-supported conclusions: interactive AV tools produce measurable learning improvements over static instruction when they promote active engagement; recursion is a known cognitive bottleneck that benefits specifically from tree-based representations; existing recursion visualization tools collectively fail to provide a purpose-built, web-accessible, step-by-step recursion tree for canonical DAA algorithms; and both Karatsuba and Maximum Subarray are pedagogically rich problems whose key insights are more naturally revealed through their recursion trees than through code or prose.

2.4.2 Critical Evaluation of the Literature

While the empirical literature is generally supportive, several methodological limitations must be acknowledged. Many studies in this domain suffer from small sample sizes, short treatment durations, and outcomes measured immediately after instruction rather than longitudinally. Hundhausen et al.'s (2002) meta-study noted considerable heterogeneity across the 24 studies it examined, limiting the precision of generalised conclusions. The AVDS study (2024), while producing impressive outcomes, was conducted in a single institutional context without a rigorous control group. Istiono's (2023) comparative analysis was published as an unreviewed preprint, and its experimental methodology is not fully specified. Existing surveys of AV tools are largely descriptive rather than comparative, relying on feature inventories rather than head-to-head empirical comparisons.

2.4.3 *Gaps and Inconsistencies*

1. Absence of a dedicated animated Karatsuba visualization tool: no existing platform provides step-by-step recursion tree animation specifically for this algorithm.
2. Deployment friction: the most pedagogically grounded recursion tool (SRec) requires Java installation; no peer-reviewed study describes a purpose-built, web-deployed, navigable recursion tree animator for divide-and-conquer DAA algorithms.
3. Lack of multi-algorithm comparative visualization: no existing tool enables direct comparison of ternary-branching (Karatsuba) and binary-branching (Max Subarray) trees within a single interface.
4. Insufficient longitudinal and transfer studies: the literature lacks studies assessing whether recursion tree visualization produces durable gains in divide-and-conquer problem-solving on novel problems.

2.5 Conclusion

2.5.1 *Summary of Key Findings*

- Algorithm visualization is empirically validated, with Hundhausen et al. (2002) confirming effectiveness specifically when tools actively engage students through step-by-step interaction and prediction tasks.
- Recursion constitutes a known cognitive bottleneck, and tree-based representations are the most effective scaffold — supported by both textbook authors (Miller and Ranum) and conference research (SIGCSE 2023).
- Existing recursion visualization tools are either too generic (VisuAlgo), code-level rather than algorithmic (Python Tutor), or require friction-inducing desktop installation (SRec).
- Karatsuba's ternary branching structure makes the $O(n^{1.585})$ complexity argument uniquely visual — the reduction from 4 sub-problems to 3 is the algorithm's central insight and is most clearly communicated through the recursion tree.
- The Maximum Subarray divide-and-conquer formulation is pedagogically indispensable as a demonstration of the $O(n \log n)$ complexity class arising from binary recursion with linear-time combining.

2.5.2 *Implications for Future Research*

The gaps identified suggest several directions: a controlled empirical study comparing learning outcomes for Karatsuba across visualization-assisted and non-visualization-assisted

instruction; extension of the tool to additional divide-and-conquer algorithms enabling multi-algorithm comparison views; longitudinal assessment studies tracking whether recursion tree visualization produces durable improvements in problem-solving on novel problems; and integration studies examining how the tool performs when explicitly scaffolded with complementary pen-and-paper recurrence derivation activities.

2.6 References

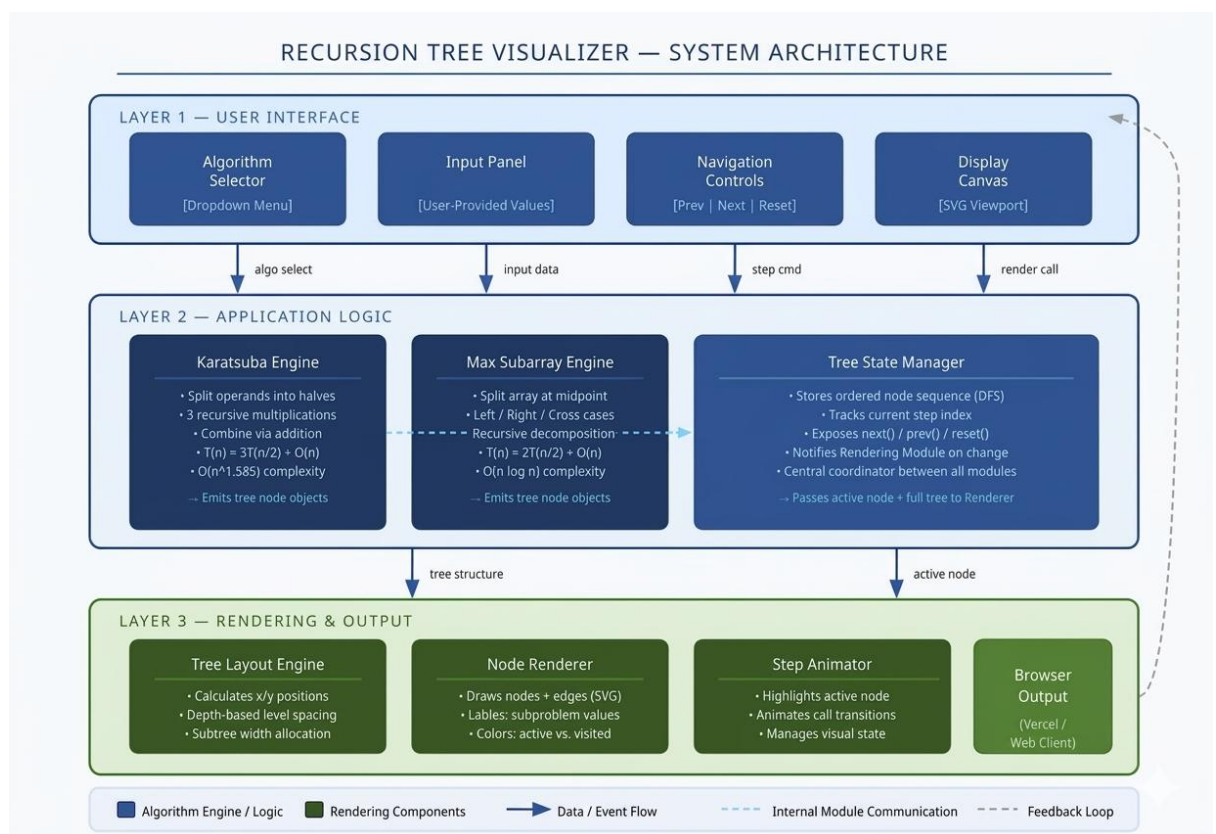
All references are formatted in IEEE style.

- [1]C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *Journal of Visual Languages and Computing*, vol. 13, no. 3, pp. 259–290, 2002.
- [2]S. Hansen, E. Schrimpscher, and N. H. Narayanan, "Designing educationally effective algorithm visualizations," *Journal of Visual Languages and Computing*, 2002. [Online]. Available: <https://www.eng.auburn.edu/~naraynh/jvlc.pdf>
- [3]J. Á. Velázquez-Iturbide, A. Pérez-Carrasco, and J. Urquiza-Fuentes, "SRec: An animation system of recursion for algorithm courses," *ACM SIGCSE Bulletin*, vol. 40, no. 3, pp. 225–229, 2008.
- [4]J. Á. Velázquez-Iturbide et al., "Automatic visualization of recursion trees," *Electronic Notes in Theoretical Computer Science*, vol. 86, no. 6, 2003.
- [5]P. J. Guo, "Online Python Tutor: Embeddable web-based program visualization for CS education," in *Proc. ACM SIGCSE Technical Symposium*, 2013, pp. 579–584.
- [6]S. Halim, VisuAlgo. National University of Singapore, 2011–present. [Online]. Available: <https://visualgo.net/>
- [7]B. Miller and D. Ranum, *Problem Solving with Algorithms and Data Structures*. Runestone Academy, 2011.
- [8]W. Istiono, "Analyzing time and space complexity: Kadane vs. divide and conquer for maximum sub-array," *SSRN*, 2023. <https://ssrn.com/abstract=4643200>
- [9]A. Karatsuba and Y. Ofman, "Multiplication of multidigit numbers on automata," *Soviet Physics Doklady*, vol. 7, pp. 595–596, 1962.
- [10]U. Grenander, *Pattern Synthesis: Lectures in Pattern Theory*. New York: Springer-Verlag, 1977.
- [11]Springer Nature, "The efficacy of animation and visualization in teaching data structures," *Educational Technology Research and Development*, 2024. <https://doi.org/10.1007/s11423-024-10382-w>
- [12]M. Andreev et al., "Use of visualization technologies in distance education," *Educational Technology Research*, 2021.

- [13]R. A. Nathasya et al., "DS-PITON: An integrated algorithm visualization tool," Proc. International Conf. on Information Technology, 2019.
- [14]P. Perháč and S. Šimoňák, "Interactive system for algorithm and data structure visualization," Computer Science Journal of Moldova, vol. 30, no. 1, pp. 28–48, 2022.
- [15]T. L. Naps et al., "Exploring the role of visualization and engagement in CS education," ACM SIGCSE Bulletin, vol. 35, pp. 131–152, 2002.
- [16]University of Chicago, CMSC-272: The Karatsuba Algorithm.
<https://people.cs.uchicago.edu/~laci/HANDOUTS/karatsuba.pdf>
- [17]MIT OpenCourseWare, 6.006 Introduction to Algorithms. <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/>
- [18]Cornell University, CS3110 – Recursion Trees and the Master Method. <https://cs3110.github.io/>
- [19]GeeksforGeeks, "Karatsuba algorithm for fast multiplication," 2023.
<https://www.geeksforgeeks.org/karatsuba-algorithm-for-fast-multiplication-using-divide-and-conquer-algorithm/>
- [20]GeeksforGeeks, "Maximum subarray sum using Divide and Conquer," 2023.
<https://www.geeksforgeeks.org/maximum-subarray-sum-using-divide-and-conquer-algorithm/>
- [21] Wikipedia, "Karatsuba algorithm," 2025. https://en.wikipedia.org/wiki/Karatsuba_algorithm
- [22]Wikipedia, Maximum subarray problem," 2025. https://en.wikipedia.org/wiki/Maximum_subarray_problem

3.0 Design Figure

The following figure presents the complete system architecture of the Recursion Tree Visualizer. The design adopts a three-layer model: User Interface, Application Logic, and Rendering that separates concerns clearly and reflects the unidirectional data flow governing the tool's operation. Each layer communicates with adjacent layers through well-defined interfaces: user events flow downward from the interface into the logic layer, while rendered output flows upward from the rendering layer back to the browser display.



3.1 Explanation of the Design Figure

The architecture diagram in Figure 1 organises the Recursion Tree Visualizer into three horizontal layers, each representing a distinct tier of the application's internal structure. Data flows vertically between layers via solid arrows, while internal module communication

within the Application Logic Layer is shown using dashed arrows. A feedback loop from the Rendering Layer back to the browser client appears on the right side of the diagram, representing the final visual output the user observes.

Layer 1 — User Interface

The topmost layer contains all components through which the user directly interacts with the tool. It comprises four modules: the Algorithm Selector (a dropdown allowing the user to choose between Karatsuba Multiplication and Maximum Subarray), the Input Panel (where the user provides operands or array contents), the Navigation Controls (Prev, Next, and Reset buttons that drive step-by-step traversal), and the Display Canvas (the SVG viewport in which the tree is rendered and animated). All user interactions originate in this layer and are passed downward as events to the Application Logic Layer.

Layer 2 — Application Logic

The middle layer is the computational core of the application. The Karatsuba Engine implements the full recursive divide-and-conquer logic for Karatsuba Multiplication splitting operands into high and low halves, performing three recursive multiplications, and combining results. The Max Subarray Engine implements the recursive decomposition for the Maximum Subarray problem splitting the input array at its midpoint and handling the left half, right half, and crossing cases at each level. Both engines generate tree node objects at each recursive call and pass them to the Tree State Manager, which is the central coordinator: it stores the full ordered sequence of recursive calls, tracks the current step index, exposes next(), prev(), and reset() to the Navigation Controls, and notifies the Rendering Layer whenever the active node changes.

Layer 3 — Rendering and Output

The bottommost layer translates abstract tree data into a visual representation. The Tree Layout Engine calculates x and y coordinates for each node based on recursion depth

and subtree width. The Node Renderer draws nodes and connecting edges onto the SVG canvas, labelling each node with its sub-problem values. The Step Animator highlights the currently active node, manages colour-coding between visited and pending states, and applies smooth transitions between steps. The output of this layer is the live browser display hosted on Vercel.

4.0 Module Description

The Recursion Tree Visualizer is composed of five distinct software modules, each with a clearly defined purpose, set of responsibilities, and set of interactions with other modules. Together these five modules implement the complete functionality of the tool from user input through algorithmic computation to animated visual output.

4.1 Module 1: Algorithm Selector

Attribute	Details
Type	UI Control Component
Interface element	Dropdown menu (HTML <select>)
Current options	Karatsuba Multiplication Maximum Subarray (Divide & Conquer)
Interacts with	Tree State Manager, Karatsuba Engine, Max Subarray Engine

The Algorithm Selector module is the entry point of the application. It presents the user with a dropdown menu listing all currently implemented algorithms. When a selection is made, the module fires a change event that performs two actions: it signals the Tree State Manager to clear all stored state and reset the step index to zero, and it activates the algorithm engine corresponding to the selected option. The module is deliberately designed as a simple, stateless UI control — it holds no data itself and delegates all state management to the Tree State Manager.

The Algorithm Selector is architecturally important because it serves as the primary switching mechanism between the two algorithm engines. Rather than maintaining two separate application instances, the tool uses a single shared infrastructure (Tree State Manager, Rendering Layer) and routes the output of whichever engine is active through this shared pipeline. This design keeps the codebase DRY (Do Not Repeat Yourself) and makes it

straightforward to add future algorithms: a developer need only implement a new engine and register it as a new option in the dropdown.

4.2 Module 2: Karatsuba Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module
Input	Two integers x and y provided via the Input Panel
Output	Ordered sequence of tree node objects (depth, operands, partial products, return value)
Time complexity	$O(n^{\log_2 3}) \approx O(n^{1.585})$, where n = digit length of operands
Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)

The Karatsuba Engine implements the full recursive logic of the Karatsuba Multiplication algorithm. When invoked with two integers x and y , it executes the following sequence at each recursion level: (1) checks whether x or y is a single digit and returns the direct product as a base case; (2) determines the split position m based on the number of digits in the larger operand; (3) splits both numbers into high and low halves: $x = x_1 \cdot 10^m + x_0$ and $y = y_1 \cdot 10^m + y_0$; (4) computes three recursive products: $z_0 = x_0 \cdot y_0$, $z_2 = x_1 \cdot y_1$, and $z_1 = (x_1 + x_0)(y_1 + y_0) - z_2 - z_0$; and (5) assembles the final result as $z_2 \cdot 10^{(2m)} + z_1 \cdot 10^m + z_0$.

At each recursive call, the engine creates a tree node object capturing the depth level, operand values, computed partial products, and return value. These node objects are passed into the Tree State Manager in depth-first order, building the full tree data structure the Rendering Layer will visualise. The ternary branching structure, three recursive calls per non-base-case node is a direct consequence of the Karatsuba identity and makes the algorithm's $O(n^{1.585})$ complexity visually explicit in the rendered tree: the branching factor of 3 versus 4 in the naïve approach is immediately observable across tree levels.

4.3 Module 3: Max Subarray Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module
Input	A one-dimensional integer array provided via the Input Panel
Output	Ordered tree node objects (subarray range, midpoint, max values per case, return value)
Time complexity	$O(n \log n)$ via recurrence $T(n) = 2T(n/2) + O(n)$
Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)

The Max Subarray Engine implements the divide-and-conquer solution to the Maximum Subarray problem. When invoked with array A and bounds low and $high$, it: (1) checks whether low equals $high$, returning $A[low]$ as the base case; (2) computes midpoint $mid = [(low + high) / 2]$; (3) recursively finds the maximum subarray in $A[low..mid]$; (4) recursively finds the maximum subarray in $A[mid+1..high]$; (5) finds the maximum crossing subarray by scanning outward from mid in both directions; and (6) returns the maximum of the three results.

A tree node object is created at each recursive call, recording the subarray range $[low, high]$, the midpoint value, the maximum values found in the left, right, and crossing cases, and the overall return value. These are emitted into the Tree State Manager in DFS order. The binary branching structure produces $\log n$ levels in the tree, making the $O(n \log n)$ recurrence directly observable from the rendered tree.

4.4 Module 4: Tree State Manager

Attribute	Details
Type	Central State Coordinator
Stores	Ordered array of all tree node objects; current step index (integer)
Exposes	<code>next()</code> , <code>prev()</code> , <code>reset()</code> , <code>getCurrentNode()</code> , <code>getFullTree()</code>

Interacts with	Both engines (receives nodes); Navigation Controls (receives commands); Rendering Layer (notifies on change)
----------------	--

The Tree State Manager is the central coordination module of the Recursion Tree Visualizer. When an algorithm engine executes, it passes each node object into the Tree State Manager, which appends them to an ordered sequence following depth-first, left-to-right traversal, the exact execution order of the recursive calls. This means stepping forward through the sequence exactly replicates the order in which recursive calls are made at runtime, giving the learner a faithful trace of the algorithm's execution.

The Tree State Manager exposes a clean interface to both the Navigation Controls and the Rendering Layer. Next increments the step index and emits a state-change notification; Prev decrements it; Reset returns the index to zero and clears all visited states. On every state change, the manager supplies the Rendering Layer with `getFullTree()` and `getCurrentNode()` data, triggering a re-render with the correct active node highlighted. This event-driven notification pattern keeps the Rendering Layer fully decoupled from the logic layer, it simply reacts to state changes without needing to know how the state was produced.

4.5 Module 5: Rendering Module

Attribute	Details
Type	Visual Output Module (three sub-components)
Sub-components	Tree Layout Engine Node Renderer Step Animator
Input	Full tree data + active node from Tree State Manager
Output	Live SVG visualisation in the browser Display Canvas
Interacts with	Tree State Manager (receives data); Display Canvas (writes SVG elements)

The Rendering Module is responsible for the complete visual output of the tool, composed of three cooperating sub-components. The Tree Layout Engine processes the full

tree structure and computes (x, y) coordinates for each node using a recursive positioning algorithm: vertical position is based on recursion depth multiplied by a fixed spacing constant; horizontal position is assigned via a post-order pass that places leaf nodes sequentially and centres each parent over its children, with a pre-order correction pass shifting overlapping subtrees rightward. The result is a balanced, symmetric layout that scales with both depth and branching factor.

The Node Renderer draws labelled SVG circle elements for each node and directed SVG line elements for each edge. Nodes are colour-coded: unvisited nodes render in neutral grey; visited nodes render in solid blue; and the currently active node renders in highlighted orange/gold to draw immediate visual attention. The Step Animator sub-component manages transitions between steps, applying smooth SVG fill-colour and stroke-width transitions so that the visual focus shift is animated rather than abrupt; a deliberate design choice that aids the learner in tracking which recursive call is currently being executed.

5.0 Dataset and Method Description

5.1 Dataset Description

The Recursion Tree Visualizer is an algorithm analysis and education tool rather than a data-driven system. It does not operate on a fixed external dataset. Instead, its input is user-defined at runtime and constitutes synthetic, user-generated data supplied interactively through the browser interface. The following subsections describe the input specification for each algorithm, framed as a dataset description in terms of source, structure, features, and preprocessing, consistent with academic reporting conventions for tools of this type.

5.1.1 Input Specification: Karatsuba Multiplication

Property	Description
Source	User-provided at runtime via the Input Panel in the browser interface
Format	Two non-negative integers x and y , entered as decimal strings
Size / Scale	Any integer of arbitrary digit length n ; recursive depth increases as $O(\log n)$
Features	Digit length n (determines recursion depth); value of each digit (determines base-case outcomes); relative sizes of x and y (determines split point m)
Preprocessing Step 1	Zero-padding: if x and y have unequal digit lengths, the shorter number is left-padded with zeros to equalise lengths before splitting, ensuring the divide step is symmetric
Preprocessing Step 2	Split computation: split position $m = n/2$ is computed, and each number is expressed as $x = x_1 \cdot 10^m + x_0$ where $x_1 = \lfloor x / 10^m \rfloor$ and $x_0 = x \bmod 10^m$
Example input	$x = 1234$, $y = 5678$ — 4-digit operands producing a recursion tree of depth 2 with $3^2 = 9$ non-base-case calls
External dataset	None. No pre-existing dataset files, repositories, or APIs are used.

5.1.2 Input Specification: Maximum Subarray (Divide and Conquer)

Property	Description
Source	User-provided at runtime via the Input Panel in the browser interface
Format	A one-dimensional array A of integers, entered as a comma-separated list
Size / Scale	Array length n ; recursion tree has exactly $(\log_2 n)$ levels with 2^k nodes at level k
Features	Array length n (determines tree depth); individual element values

	(determine base-case returns and crossing sums); sign distribution (positive/negative mix determines the structure of the maximum subarray)
Preprocessing Step 1	Index binding: algorithm initialised with low = 0 and high = n – 1, providing explicit boundary indices that drive all recursive splits
Preprocessing Step 2	Midpoint computation: at each recursive call, mid = [(low + high) / 2] is computed and stored as a node attribute, visible in the tree label
Example input	A = [–2, 1, –3, 4, –1, 2, 1, –5, 4] — n = 9, answer = 6, subarray [4, –1, 2, 1]
External dataset	None. No pre-existing dataset files, repositories, or APIs are used.

5.2 Method Description

This section describes the two algorithmic methods implemented, the rationale for their selection, the tree generation method used to produce the step sequence, and the rendering method used to lay out and display the tree. Together, these methods constitute the complete computational and visual pipeline of the application.

5.2.1 Method 1: Karatsuba Multiplication

Karatsuba Multiplication is a divide-and-conquer algorithm for multiplying two n-digit integers, first published by Anatoly Karatsuba in 1962. It is the canonical example of achieving sub-quadratic complexity through algebraic cleverness. Given $x = x_1 \cdot B^m + x_0$ and $y = y_1 \cdot B^m + y_0$ where B is the numeric base and $m = (n/2)$, the product xy can be expressed as:

$$xy = z \cdot B^{(2m)} + z_1 \cdot B^m + z_0$$

where $z_0 = x_0y_0$, $z_2 = x_1y_1$, and $z_1 = (x_1 + x_0)(y_1 + y_0) - z_2 - z_0$. The critical insight is that z_1 requires only one multiplication — since z_0 and z_2 have already been computed reducing four multiplications to three. This yields the recurrence $T(n) = 3T(n/2) + O(n)$, which resolves by the Master Theorem ($a=3, b=2, \log_b a \approx 1.585 > 1$) to $T(n) = O(n^{1.585})$.

Property	Naïve Multiplication	Karatsuba
----------	----------------------	-----------

Subproblems per call	4 ($x_1y_1, x_1y_0, x_0y_1, x_0y_0$)	3 ($z_0, z_2, (x_1+x_0)(y_1+y_0)$)
Recurrence	$T(n) = 4T(n/2) + O(n)$	$T(n) = 3T(n/2) + O(n)$
Complexity	$O(n^2)$	$O(n^{1.585})$
Tree branching factor	4 (quaternary tree)	3 (ternary tree)

Karatsuba Multiplication was selected because its complexity improvement over the naïve approach is directly expressed in the recursion tree: the branching factor of 3 rather than 4 is the only structural difference, and its impact reducing the exponent from 2 to 1.585 is immediately observable from tree depth and node count. No other algorithmic representation makes this connection as transparent.

5.2.2 Method 2: Maximum Subarray (Divide and Conquer)

The Maximum Subarray divide-and-conquer approach proceeds as follows. Given array A with bounds $[low, high]$: (1) if $low = high$, return $A[low]$; (2) compute $mid = [(low + high) / 2]$; (3) recursively find $maxLeft = \text{MaxSubarray}(A, low, mid)$; (4) recursively find $maxRight = \text{MaxSubarray}(A, mid+1, high)$; (5) compute $maxCross$ by scanning outward from mid in both directions; (6) return $\max(maxLeft, maxRight, maxCross)$. The recurrence $T(n) = 2T(n/2) + O(n)$ resolves by the Master Theorem (Case 2, $a=2, b=2, \log_b a = 1$) to $T(n) = O(n \log n)$.

Property	Divide & Conquer	Kadane's Algorithm
Approach	Recursive decomposition	Dynamic programming (linear scan)
Recurrence	$T(n) = 2T(n/2) + O(n)$	No recurrence — iterative
Time complexity	$O(n \log n)$	$O(n)$
Space complexity	$O(\log n)$ — call stack	$O(1)$
Recursion tree generated?	Yes — binary tree, $\log n$ levels	No
Pedagogical purpose	Demonstrates $O(n \log n)$ class visually	Optimal for production use

The divide-and-conquer formulation was chosen over Kadane's $O(n)$ algorithm for an explicit pedagogical reason: Kadane's algorithm is a single linear pass with no recursive structure, and therefore generates no recursion tree. The divide-and-conquer approach, while algorithmically suboptimal, produces a clean binary recursion tree whose $O(n \log n)$ complexity is directly visible in $\log n$ depth levels and binary branching. Placed alongside Karatsuba's ternary tree, the two algorithms together allow students to compare directly how branching factor and recurrence structure determine the complexity class.

5.2.3 Tree Generation Method

Both algorithm engines use the same tree generation method: they execute the recursive algorithm normally but instrument each recursive function call to produce a tree node object before returning. Each node object captures: a unique node ID; the recursion depth level (0 = root); the parent node ID (null for root); the input parameters at that call; the intermediate values computed at that level; and the return value. Nodes are appended to an ordered list maintained by the Tree State Manager in the order they are created: depth-first, left-to-right, corresponding exactly to actual execution order.

This DFS ordering is significant for the step navigator: clicking Next advances exactly one recursive call in execution order, and clicking Prev retreats one call. The learner is therefore tracing the algorithm in the same order the computer executes it, rather than a simplified or reordered version. The full ordered sequence constitutes the tree's DFS traversal, from which the Rendering Module recovers the parent-child tree topology by following parent node ID references.

5.2.4 Rendering and Layout Method

The Tree Layout Engine uses a Reingold-Tilford-inspired recursive approach to compute node positions. Each node is assigned a y-coordinate equal to its depth multiplied by a fixed vertical spacing constant (approximately 80–100 pixels). Horizontal x-coordinates are

assigned by a post-order pass: leaf nodes are placed at sequential horizontal positions, and each internal node is centred over its children. A pre-order correction pass then shifts any overlapping subtrees rightward, ensuring no two nodes at the same depth occupy the same horizontal position. The result is a balanced, symmetric tree layout that scales naturally with the depth and branching factor of the algorithm.

The Node Renderer writes SVG circle elements for each node and SVG line elements for each edge. The Step Animator applies SVG fill-colour transitions and stroke-width changes to distinguish the active node (highlighted orange), visited nodes (solid blue fill), and pending nodes (outlined only, grey fill). On each state change emitted by the Tree State Manager, the Rendering Module re-evaluates the active node, updates the relevant SVG attributes, and applies the transition animation, producing the step-by-step animated effect that is central to the tool's educational purpose.

6.0 Conclusion

The Recursion Tree Visualizer was developed in response to a clearly identified gap in algorithm education: the absence of a purpose-built, web-accessible, step-by-step animated recursion tree tool for the canonical divide-and-conquer algorithms taught in undergraduate DAA curricula. This report has documented the project across five dimensions: its introduction and objectives, its literature foundations, its system architecture, its internal module structure, and its algorithmic and methodological design. This concluding section draws together the key outcomes of the project, reflects on its limitations, and outlines directions for future development.

6.1 Summary of Achievements

The project successfully delivers on all five of its stated objectives. The tool renders the step-by-step execution of both Karatsuba Multiplication and the Maximum Subarray divide-and-conquer algorithm as fully animated, navigable recursion trees, accessible through any modern web browser without installation. The recursive structure of each algorithm — including its branching factor, depth, subproblem decomposition, and return value propagation is made visually explicit at every step. The step-by-step navigation system, driven by the Tree State Manager, gives learners granular control over the pace and direction of their exploration, directly addressing the research finding that active engagement with visualizations is the primary driver of improved learning outcomes.

From an architectural standpoint, the three-layer design (User Interface, Application Logic, Rendering) achieves clean separation of concerns and a DRY codebase that is straightforwardly extensible. The two algorithm engines share the same tree generation and rendering infrastructure, meaning that new algorithms can be added by implementing a new engine and registering it in the Algorithm Selector, with no changes required to the rest of the

system. This design decision reflects not only good software engineering practice but also the tool's long-term ambition to become a multi-algorithm reference platform for DAA coursework.

The pairing of Karatsuba Multiplication (ternary branching, $O(n^{1.585})$) with the Maximum Subarray divide-and-conquer algorithm (binary branching, $O(n \log n)$) is a deliberate pedagogical choice that pays off in the visualized output: placing the two trees side by side makes the relationship between branching factor and complexity class directly observable, providing a level of comparative insight that no static textbook figure, no pseudocode listing, and no recurrence calculation alone can match. This is the core educational contribution of the project.

6.2 Limitations

Several limitations of the current implementation should be acknowledged. First, the tool currently supports only two algorithms. While these two were chosen for their pedagogical complementarity, a richer platform would include additional divide-and-conquer algorithms :Merge Sort, Quick Sort, Strassen's Matrix Multiplication, the Tower of Hanoi, to allow students to build a broader comparative understanding of recursive complexity classes. Second, the tool does not currently include any in-interface explanation or annotation beyond node labels. A future version could incorporate tooltips, collapsible text panels at each node explaining what sub-problem is being solved, or a side-by-side recurrence equation view that updates as the student navigates. Third, the tool has not yet been formally evaluated in a classroom setting. While its design is grounded in well-established empirical literature on algorithm visualization, no controlled study has been conducted measuring its specific impact on student comprehension of Karatsuba or Maximum Subarray, a gap that limits the claims that can be made about its effectiveness.

A further limitation is the absence of persistent session state. If a student refreshes the browser mid-walkthrough, the current position in the recursion tree is lost and the visualization resets. For a study tool intended for repeated use across sessions, some form of lightweight state persistence would significantly improve the user experience.

6.3 Future Work

The most immediate planned extension is the addition of further divide-and-conquer algorithms, with Merge Sort and the Fibonacci sequence being the highest priority. Merge Sort would add a third data point to the branching-factor comparison (binary branching, but with a different $O(n \log n)$ derivation from Max Subarray due to its guaranteed rather than input-dependent structure), while Fibonacci would introduce the concept of overlapping subproblems and exponential branching providing a natural bridge to dynamic programming as the next topic in a typical DAA course.

A second planned extension is the addition of an in-interface complexity analysis panel that renders the recurrence relation for the active algorithm, applies the Master Theorem step-by-step alongside the tree, and highlights which level of the tree contributes the dominant term to the total work. This would directly connect the visual tree structure to the formal mathematical analysis that students are expected to reproduce in examinations.

Longer-term, a formal classroom evaluation study, ideally a pre-post design comparing student performance on recursion tree problems before and after using the tool, with a matched control group would fill the most significant gap identified in the literature review and provide the empirical evidence needed to advocate for the tool's integration into DAA curricula beyond this project's institutional context. Collaboration with the course instructor to embed the tool as a required pre-lecture activity, paired with pen-and-paper recurrence derivation exercises, would also allow investigation of the scaffolding question

raised in the literature: whether visualization alone produces deep abstract reasoning, or whether it must be paired with complementary analytical tasks to achieve durable learning gains.

6.4 Closing Remarks

The Recursion Tree Visualizer demonstrates that a focused, purpose-built educational tool, one that does fewer things than a general-purpose platform but does them with greater depth and pedagogical intentionality can address a genuine gap in the algorithm education ecosystem. By making the execution of two of the most instructive divide-and-conquer algorithms in the DAA curriculum visible, navigable, and analytically transparent, the tool provides a learning experience that complements and extends what textbooks, lectures, and static diagrams can offer. The project is a working proof of concept for this approach, and the architecture it establishes provides a solid foundation for the more expansive platform it is designed to grow into.